

Contents

1	前置作業	
1.1	浮點數	
2	資料結構	
2.1	BIT	
3	字串	
3.1	字典樹 trie	
3.2	LCP	
3.3	KMP	
4	圖論	
4.1	建點	
4.2	點直線、直線直線關係	
4.3	凸包問題	
5	數學	
5.1	定理	

1 前置作業

1.1 浮點數

```
1 #define eps 1e-6
2 int sgn(double x){return (x>eps)-(x<-eps);}
3 // return (-1,0,1)->(負數,0,正數)
```

2 資料結構

2.1 BIT

```
1 struct Binary_Indexed_Tree{//一定要用 1-base
2     int n;
3     vector<int> bit;
4     void init(int _n){
5         n=_n+1;
6         bit=vector<int>(n,0);
7     }
8     int lowbit(int x){return x&-x;}
9     void update(int x,int v){
10         for(;x<n;x+=lowbit(x)){
11             bit[x]+=v;
12         }
13     void range_update(int l,int r,int v){
14         update(l,v);
15         update(r+1,-v);
16     }
17     int qry(int x){
18         int ans=0;
19         for(;x>0;x-=lowbit(x)){
20             ans+=bit[x];
21         }
22         return ans;
23 };
```

3 字串

3.1 字典樹 trie

```
1 struct trie{
2     trie *nxt[26]={};
3     int cnt=0;
4     int sz=0;
5 };
6 trie *root=new trie();
7 void insert(const string &s){
8     trie *now=root;
9     for(auto i:s){
10         if(now->nxt[i-'a']==NULL){
11             now->nxt[i-'a']=new trie();
12         }
13         now=now->nxt[i-'a'];
14 }}
```

```
15 int qry(string &s){
16     trie *now=root;
17     for(auto i:s){
18         if(now->nxt[i-'a']==NULL) return;
19         now=now->nxt[i-'a'];
20     }
21     return now->cnt;
22 }
```

3.2 LCP

```
1 int lcp(const string &pa,const string &pb){//字串先排序
2     int i=0;
3     while(1<pa.size() && i<pb.size() && pa[i]==pb[i])
4         i++;
5     return i;
6 }
```

3.3 KMP

```
1 vector<int> kmp(string s,string t){
2     if(t.size()>s.size()) return {};
3     vector<int> pi(t.size(),0);
4     for(int i=1,j=0;i<t.size();i++){
5         while(j>0 && t[j]!=t[i]){
6             j=pi[j-1];
7         }
8         if(t[j]==t[i]) j++;
9         pi[i]=j;
10    }
11    vector<int> ans;
12    for(int i=0,j=0;i<s.size();i++){
13        while(j>0 && t[j]!=s[i]){
14            j=pi[j-1];
15        }
16        if(t[j]==s[i]) j++;
17
18        if(j==t.size()){
19            ans.push_back(i-j+1);
20            j=pi[j-1];
21        }
22    }
23    return ans;
24 }
```

4 圖論

4.1 建點

```
1 template<class T>
2 struct pt{
3     T x,y;
4     pt(T _x,T _y):x(_x),y(_y){};
5     pt():x(0),y(0){};
6
7     pt operator+(pt a){return pt(x+a.x,y+a.y);}
8     pt operator-(pt a){return pt(x-a.x,y-a.y);}
9     pt operator*(T c){return pt(x*c,y*c);}
10    pt operator/(T c){return pt(x/c,y/c);}
11
12    T operator*(pt a){return x*a.x+y*a.y;}
13    T operator^(pt a){return x*a.y-y*a.x;}
14
15    bool operator<(const pt &a) const{
16        return x<a.x || (x==a.x && y<a.y);
17    }
18 };
19 using Pt=pt<int>;
```

4.2 點直線、直線直線關係

```

1 auto cross(Pt a,Pt b,Pt c){return (b-a)^(c-a);}
2 auto dot(Pt a,Pt b,Pt c){return (b-a)*(c-a);}
3 struct Line{Pt a,b;};
4 bool PtOnSeg(Pt p,Line l){ //判斷在線段上
5     return sgn(cross(l.a,l.b,p))==0 &&
6         sgn(dot(p,l.a,l.b))<=0;
7 }
8 double PtSegDist(Pt p,Line l){ //求點線距
9     double ans=min(abs(p-l.a),abs(p-l.b));
10    if(sgn(dot(l.a,l.b,p))<0) return ans;
11    if(sgn(dot(l.b,l.a,p))<0) return ans;
12    return min(ans,
13        abs(cross(p,l.a,l.b))/abs(l.a-l.b));
14 }
15 int PtSide(Pt p,Line l){
16     return sgn(cross(l.a,l.b,p));
17 }
18 bool isInter(Line l1,Line l2){ //線段相交
19     if(PtOnSeg(l2.a,l1) || PtOnSeg(l2.b,l1) ||
20        PtOnSeg(l1.a,l2) || PtOnSeg(l1.b,l2))
21         return 1;
22     return PtSide(l2.a,l1)*PtSide(l2.b,l1)<0 &&
23        PtSide(l1.a,l2)*PtSide(l1.b,l2)<0;
24 }

```

4.3 凸包問題

```

1 struct ConvexHull{
2     vector<Pt> vec;
3     int n;
4     void init(vector<Pt> &v){
5         vec=v;
6         n=v.size();
7     }
8     int cross(Pt a,Pt o,Pt b){
9         return (a-o)^(b-o);
10    }
11    vector<int> point;
12    vector<int> area;
13    void build(){
14        point=vector<int>(n+1);
15        area=vector<int>(n+1);
16        vector<Pt> up,down;
17        int upsum=0;
18        int downsum=0;
19        for(int i=0;i<n;i++){
20            while(up.size()>=2 &&
21                cross(up[up.size()-2],
22                    up[up.size()-1],
23                    vec[i])<=0){
24                upsum-=cross(up[up.size()-2],
25                    {0,0},up[up.size()-1]);
26                up.pop_back();
27            }
28            up.push_back(vec[i]);
29            if(up.size()>1)
30                upsum+=cross(up[up.size()-2],
31                    {0,0},
32                    up[up.size()-1]);
33
34            while(down.size()>=2 &&
35                cross(down[down.size()-2],
36                    down[down.size()-1],
37                    vec[i])>=0){
38                downsum-=cross(down[down.size()-2],
39                    {0,0},down[down.size()-1]);
40                down.pop_back();
41            }
42            down.push_back(vec[i]);
43            if(down.size()>1)
44                downsum+=cross(down[down.size()-2],
45                    {0,0},down[down.size()-1]);
46
47            area[i+1]=abs(upsum-downsum);
48
49            if(i+1==1) point[i+1]=1;
50            else point[i+1]=up.size()+down.size()-2;
51        }
52    }
53    double getarea(int p){
54        return area[p]/2.0;
55    }
56    int getnum(int p){
57        return point[p];
58    }
59 };

```

```

38         vec[i])>=0){
39             downsum-=cross(down[down.size()-2],
40                 {0,0},down[down.size()-1]);
41             down.pop_back();
42         }
43         down.push_back(vec[i]);
44         if(down.size()>1)
45             downsum+=cross(down[down.size()-2],
46                 {0,0},down[down.size()-1]);
47
48         area[i+1]=abs(upsum-downsum);
49
50         if(i+1==1) point[i+1]=1;
51         else point[i+1]=up.size()+down.size()-2;
52     }
53     double getarea(int p){
54         return area[p]/2.0;
55     }
56     int getnum(int p){
57         return point[p];
58     }
59 };

```

5 數學

5.1 定理

- 錯排公式: (n 個人中, 每個人皆不再原來位置的組合數):
 $dp[0] = 1; dp[1] = 0;$
 $dp[i] = (i - 1) * (dp[i - 1] + dp[i - 2]);$
- 一個環上相鄰顏色不同的染色數量:
 $f(n, k) = (k - 1)^n + (-1)^n (k - 1)$
 n 是點的數量, k 是顏色的數量
- König 定理:
在二分圖中
最大匹配 = 最小點覆蓋
最小邊覆蓋 = 最大獨立集
點數 = 最大匹配 + 最小邊覆蓋
點數 = 最大獨立集 + 最小點覆蓋

[illegible]